Kate Lee

March 4, 2026

IT FDN 110 GP

Assignment 06

Classes, Functions, and Separation of Concerns

Introduction

This week we took our Course Registration Program and upgraded it from a single, all-in-one script into something that looks and behaves more like real code in the wild. Instead of mixing user input, file logic, and processing logic in one giant loop, I learned to reorganize my code so that the program uses functions and classes that follow the Separation of Concerns (SoC) pattern outlined in this week's lessons (University of Washington, 2026).

Per the assignment instructions, the program had to keep the same menu behavior as Assignment05, but add functions, classes, and `@staticmethod` decorators to separate the work into clearer layers. The finished code has 3 layers (or 4 if you count declarations and imports up top). First is the processing layer, which handles reading and writing to a JSON file. Second is the presentation layer, which shows things like error messaging and the interactive parts like input and print statements. And third is the main body layer, which is kind of like the conductor of an orchestra, but the musicians are all the functions we just made.

Steps I Took

1. Reviewed Starter Code and Planned the Separation of Concerns

The module notes describe SoC as dividing a program into distinct layers so changes in one area are less likely to break another.

Last week, the code we wrote was organized rather like this, in one big chunk:

```
main loop
 ├─ input
 ├─ printing
 ├─ validation main loop
 ├─ input
 ├─ printing
```

This week, our code is organized more like this:

```
FileProcessor
 ├─ read_data_from_file()
 └─ write_data_to_file()
IO
 ├─ output_menu()
 ├─ input_menu_choice()
 ├─ input_student_data()
 └─ output_student_courses()
Main Program
 └─ coordinates everything
```

2. Created Processing and Presentation Classes

I created two classes required by the acceptance criteria: FileProcessor (for the processing layer) and IO (for the presentation layer). Each class includes a descriptive docstring, and each method uses the `@staticmethod` decorator and also has a docstring. Here's an example from my code:

```python
# -- Presentation -- #
class IO:
    """
    A collection of presentation layer functions that manage user input and output.
    """
    @staticmethod
    def output_error_messages(message: str, error: Exception = None):
        """
        Display a custom error message (and optional technical details)

        :param message: str description of what went wrong (friendly)
        :param error: optional technical Exception error info
        :return: None
        """
```

3. Implemented Required Functions (with Parameters)

I implemented the required method names and parameters exactly as listed in the assignment instructions. Using parameters makes it clearer what data a function needs and helps reduce unintended side effects compared to relying only on global variables. We also learned a little about scope, like how if you have a global variable and a local variable (inside a code block,

usually a function) that have the same name, the local/inner one is the one that gets used, unless you explicitly tell Python to use the global version of that variable.

4. Added Structured Error Handling (Try-Except-Finally)

I added structured error handling in four required places: reading the file on startup, validating first name input, validating last name input, and writing data back to the JSON file. I also tried adding a validation for course name, because I didn't want a user to just hit enter accidentally, and thereby set the course name as an empty string. So, I made sure to raise an error in that case:

```
        try:
            course = input("Please enter the name of the course: ").strip()
            if course == "":
                raise ValueError("The course name should not be empty.")
            return {"FirstName": first_name,
                    "LastName": last_name,
                    "CourseName": course}
        except Exception as e:
            IO.output_error_messages( message: "Error: Invalid course.", e)
            return None
```

Next, I worked on presenting errors consistently by making the IO class and its methods. I used the output_error_messages() method so errors are presented consistently. I like how this using this pattern reduces the lines of code you have to look through in the main body section. Making the error handling bits all live together makes a lot of sense, especially since there can be so many different kinds of errors, and it can take up a lot of space. The collapse lines icon (or whatever it's called in PyCharm?) is a really nice feature for the same reason. I usually just like looking at less stuff at once.

5. Verified Output Formatting (Comma-Separated Display)

For menu choice 2 and after saving (menu choice 3), the program displays each registration row as comma-separated values. Even though data is stored as JSON, printing CSV-style output keeps the display consistent with earlier versions of the project and the assignment's output requirements. It's also good practice for using f strings.

6. Tested in 2 IDEs

After testing in PyCharm first where I was writing my code, I also ran the script via IDLE to confirm it behaves the same.

Source Control

GitHub Repository Link:
https://github.com/katherinelee703/IntroToPython/tree/main/Assignment06

Summary

This assignment helped me understand how functions and classes can turn a growing script into something more readable and maintainable. The finished program loads existing registrations from Enrollments.json (if there are any), lets the user add more entries, displays the current registrations, and saves updates back to the JSON file. I am excited to see further iteration on the Separation of Concerns concept as we start creating more complex projects!

Works Cited

OpenAI. (2026). ChatGPT (GPT-5.2 Thinking) used for code troubleshooting and written outline assistance.

University of Washington. (2026). Introduction to Programming with Python: Module 06 Notes.